FACULTY OF ENGINEERING AND TECHNOLOGY

BACHELOR OF TECHNOLOGY

HIGH PERFORMANCE COMPUTING LABORATORY

(303105356)

VI SEMESTER

COMPUTER SCIENCE AND ENGINEERING DEPARTMENT

LABORATORY MANUAL

CERTIFICATE

This is to certify, that Mr. with enrolment no. has successfully completed his/her laboratory experiments in the **HIGH PERFORMANCE COMPUTING LAB (303105356)** from the department of COMPUTER SCIENCE AND ENGINEERING during the academic 2024-25.

Date of submission: ……. staff in charge: …….

Head of department:……

INDEX

SR. NO	TITLE	PAGE NO.		START DATE	END DATE	SIGN	MARKS
		FROM	TO				
1	Study the facilities provided by Google Colab.						
2	Demonstrate basic Linux Commands						
3	Using Divide and Conquer Strategies design a class for Concurrent Quick Sort using C++.						
4	**w**rite a program on an unloaded cluster for several different numbers of nodes and record the time taken in each case. Draw a graph of execution time against the number of nodes						
5	Write a program to check task distribution using Gprof.						
6	Use Intel V-Tune Performance Analyzer for Profiling.						
7	Analyze the code using Nvidia Profilers.						
8	Write a program to perform load distribution on GPU using CUDA.						
9	Write a simple CUDA program to print “Hello World!”						
10	Write a CUDA program to add two arrays.						

PRATICAL – 1

Aim: study facilities provided by google colab

1. **what is google colab**

Google Colab, or Colaboratory, is a free browser-based platform that lets you write and run Python code. It's hosted on Google's cloud servers, so you can use their hardware, including GPUs and TPUs, even if you don't have a powerful computer of your own. Colab is especially well-suited for machine learning, data science, and education

Here are some features of Colab:

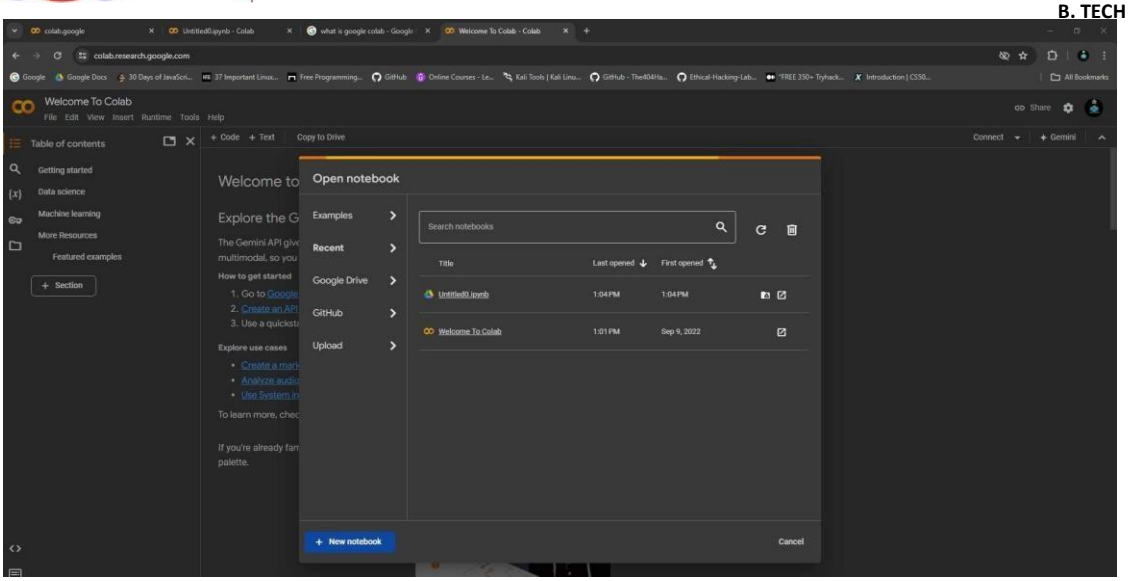
- No configuration required
- Free access to GPUs
- Easy sharing
- Leverages Google hardware
- Based on Jupyter notebooks, a popular way to write and share code

To use Colab, you must have a Google account. You can create a workbook by clicking New notebook from the File menu and inserting Python code to execute in a code cell

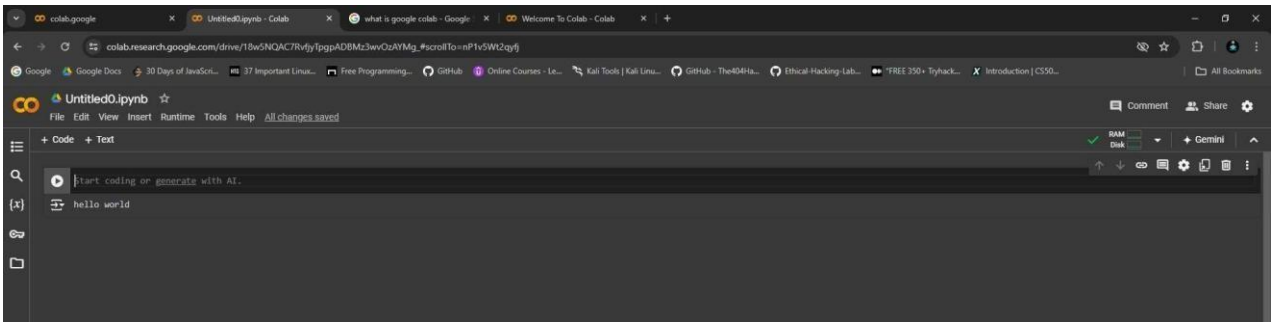
Colab is a hosted Jupyter Notebook service that requires no setup to use and provides free of charge access to computing resources, including GPUs and TPUs. Colab is especially well suited to machine learning, data science, and education.

Steps for using Google colab:

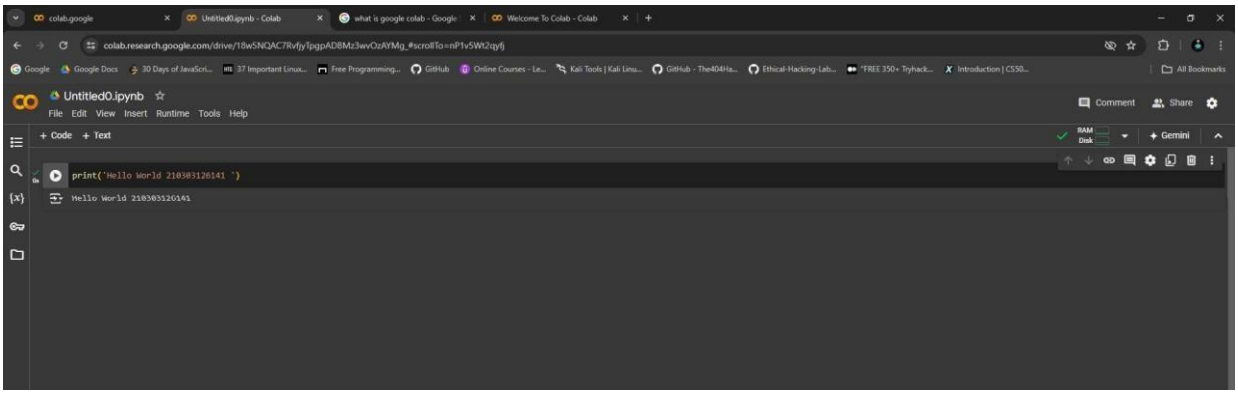
1 open any browser and type the google colab in the search bar and hit enter and now open the first link of it



2 after opening the tab in chrome an new interface will be opened and there will be option like open a new note book click on it and it will redirect to the another page



3 in the shell column type any code in it suppose I am typing the simple code "**Hello world**" of the python program after you click on the button which is left side of it . if it is first time the google colab will install necessary libraries and packages in it .



2. **features of GC**

1. Real-time collaboration: Google Colab allows multiple users to work on the same notebook simultaneously. Users can see each other's cursors and edits in real-time, enabling seamless collaboration.
2. GPU acceleration: Google Colab provides access to a powerful GPU runtime environment, allowing users to run computations on GPUs for faster processing of large datasets and deep learning models.
3. Easy integration with Google Drive: Colab integrates seamlessly with Google Drive, allowing users to store, share, and version control their notebooks and data.
4. Built-in version control: Colab automatically tracks changes made to notebooks, allowing users to revert to previous versions or create checkpoints.
5. Rich library support: Colab comes with a wide range of pre-installed libraries and packages, including popular data science libraries like NumPy, Pandas, and SciPy, as well as deep learning frameworks like TensorFlow and PyTorch.
6. Interactive visualizations: Colab supports interactive data visualization using libraries like Matplotlib and Seaborn, allowing users to create dynamic plots and graphs within the notebook.
7. Easy sharing and publishing: Users can easily share their Colab notebooks with others by generating a link or embedding the notebook in a web page. Colab also supports exporting notebooks to various formats for easier sharing and publication.

3. advantages of GC

1. Interactive and Dynamic: Google Colab allows users to write and execute code interactively within the browser, providing a dynamic and interactive environment for experimentation and exploration.
2. Collaborative Nature: Colab enables multiple users to work on the same notebook simultaneously, facilitating real-time collaboration and sharing of code, data, and results.
3. GPU Acceleration: Google Colab provides access to powerful GPUs, which can significantly speed up the training of deep learning models and complex computations.
4. Ecosystem Integration: Colab seamlessly integrates with other Google services, such as Google Drive, which allows users to store, share, and version control their notebooks and data.
5. Integrated Development Environment (IDE): Colab provides a complete Integrated Development Environment (IDE) with features like code completion, syntax highlighting, and error checking, which enhances the coding experience.

4. Disadvantages of GC

1. Limited Computational Resources: Google Colab provides limited computational resources compared to local environments or cloud-based platforms like AWS or Azure. This can lead to slower processing times and may not be suitable for computationally intensive tasks.
2. Data Storage and Persistence: Colab does not provide persistent storage for large datasets or files. Users need to rely on Google Drive or other cloud storage solutions to store and access their data.
3. Network Connectivity: Google Colab relies on a stable internet connection to access the computational resources and data. If there is a network interruption, the session may be terminated, and data may be lost.
4. Limited Customization: Colab provides a pre-configured environment with certain libraries and packages. Users may need to install additional libraries or customize the environment to suit their specific needs, which can be challenging.
5. Privacy and Security: Google Colab's environment is public and shared with other users. Any sensitive data or code should be handled with caution to ensure privacy and security.
6. Limited GPU Utilization: Colab's GPU resources are shared among multiple users, which can lead to slower GPU utilization for interactive sessions.

5. language supported by GC

Google Colab supports various programming languages, including:

1. Python: Python is the most popular language for data science, machine learning, and AI development. Colab provides a Python runtime environment that allows users to write and execute Python code within the browser.

2. R: R is a programming language specifically designed for statistical computing and graphics. Colab supports R through the use of the R runtime environment, allowing users to write and execute R code within the browser.

3. Julia: Julia is a high-level, high-performance programming language designed for numerical computing and data science. Colab supports Julia through the use of the Julia runtime environment, allowing users to write and execute Julia code within the browser.

4. MATLAB: MATLAB is a programming language used for numerical computing, visualization, and data analysis. Colab supports MATLAB through the use of the MATLAB runtime environment, allowing users to write and execute MATLAB code within the browser.

5. C++: C++ is a general-purpose programming language that can be used for various applications, including data science and AI development. Colab supports C++ through the use

of the C++ runtime environment, allowing users to write and execute C++ code within the browser.

6. Java: Java is a widely used programming language for building enterprise-level applications. Colab supports Java through the use of the Java runtime environment, allowing users to write and execute Java code within the browser.

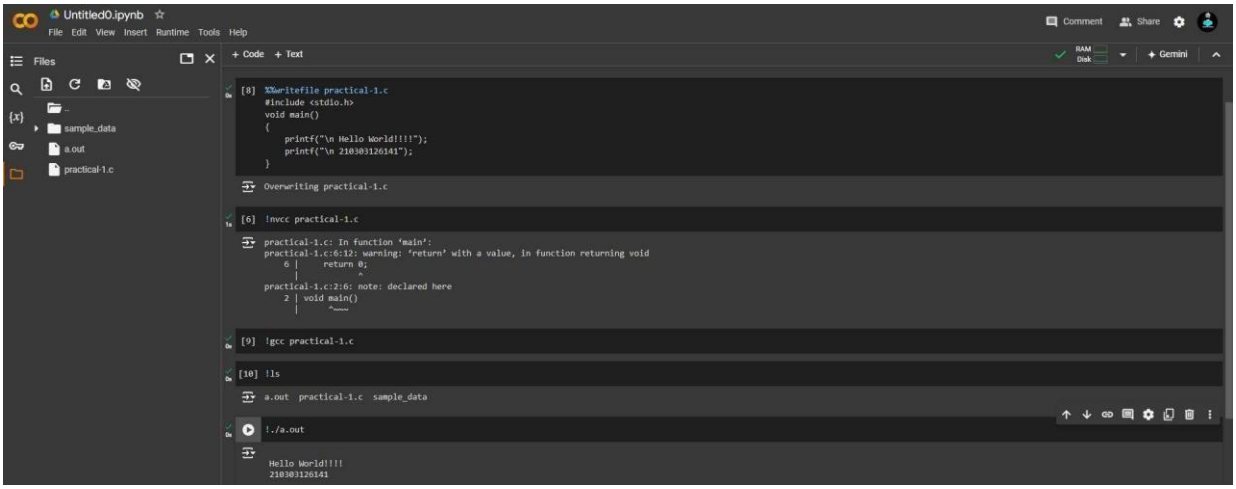
7. Scala: Scala is a functional programming language that runs on the Java Virtual Machine (JVM). Colab supports Scala through the use of the Scala runtime environment, allowing users to write and execute Scala code within the browser.

8. JavaScript (Node.js): JavaScript (Node.js) is a runtime environment that allows running JavaScript code on the server-side. Colab supports JavaScript (Node.js) through the use of the JavaScript (Node.js) runtime environment, allowing users to write and execute JavaScript code within the browser.

Execution of C programming in google colab

Steps :

1. Open a new notebook in google colab and write a simple code as we have written the Helloworld code a
2. %%writeFile practical-1.c this command is used to create a file in the colab
3. After creation of the file and we have enter **gcc <filename>**
4. GCC means The GNU Compiler Collection (GCC) is an optimizing compiler produced by the GNU Project supporting various programming languages, hardware architectures and operating systems.
5. After it is successfully compiles then another file which will be created automatically in the google colab and then we can see it in the left named as the a.out
6. For executing that file in the colab we have to run the command **./a.out** for execution and there we see the output



PRACTICAL – 2

Aim : Demonstrate basic Linux Commands

1. **Pwd**

It means Present Working Directory

Syntax : !pwd

Description : This command is useful for us to see the on which directory or file we are working. Use the pwd command to find our way in the linux file system structure maze . We use this command several times to see the directory.

```
[2] !pwd
/content
```

2. **Ls**

It means list only directory files

Syntax : !ls

Description : The LS command is one of the most commonly used commands in daily linux operations. This command is used in listing contents inside a directory

```
[8] !ls
3.txt  sample_data
```

3. **Cd**

It means the Changing Directory

Syntax : !cd

Description : CD command in linux is useful for the changing the directory of one directory to another in running program.

```
[ ] !cd revanth

[ ] !cd /revanth
/bin/bash: line 1: cd: /revanth: No such file or directory
```

4. Nano

Syntax : !nano <filename>

Description : nano is a user-friendly text editor for Unix-like systems, which allows users to create and edit files directly in the terminal. It's widely used for its simplicity and ease of use.

```
[12] !apt-get install nano

Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Suggested packages:
  hunspell
The following NEW packages will be installed:
  nano
0 upgraded, 1 newly installed, 0 to remove and 45 not upgraded.
Need to get 280 kB of archives.
After this operation, 881 kB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu jammy/main amd64 nano amd64 6.2-1 [280 kB]
Fetched 280 kB in 0s (699 kB/s)
Selecting previously unselected package nano.
(Reading database ... 121925 files and directories currently installed.)
Preparing to unpack .../archives/nano_6.2-1_amd64.deb ...
Unpacking nano (6.2-1) ...
Setting up nano (6.2-1) ...
update-alternatives: using /bin/nano to provide /usr/bin/editor (editor) in auto mode
update-alternatives: using /bin/nano to provide /usr/bin/pico (pico) in auto mode
Processing triggers for man-db (2.10.2-1) ...

[13] !nano 3.txt

>Received SIGHUP or SIGTERM

Buffer written to 3.txt.save
```

5. Cat

It means Concatenate

Syntax : !cat

Description : it is very frequently used in linux . it reads the data from the file and gives their content as output . it helps us to create , view , concatenate files.

```
[15] !cat 3.txt.save

hi hello 12 34 23 43 45 233 2344 2345 556 3566 43456 4545 43625 234526 235246 24524
```

6. Echo

Syntax : !echo

Description : The command in linux which is used to display line of text / string that are passed as an argument. This is a built in command that is mostly used in shell scripts and batch files to output text to the screen or a file

```
[ ] !echo "hello"
hello
```

7. Touch

Syntax : !touch <filename>

Description : The touch command is used to create an empty file or update the timestamp of an existing file. It is a simple yet powerful way to quickly create files in the terminal.

```
[25] !touch 4.txt
[26] !ls
3.txt  3.txt.save  4.txt  sample_data
```

8. Head

Syntax : !head abc.txt

Description : The linux head command prints the first lines of one or more files to standard output. If we want 10 lines as output in the linux we use HEAD command to program

```
[16] !head 3.txt.save
hi hello 12 34 23 43 45 233 2344 2345 556 3566 43456 4545 43625 234526 235246 24524
```

9. Tail

Syntax : !tail abc.txt

Description : It is the complementary of head command. The tail command, as the implies, print the last 10 lines of data of the given input. If more than one file name is provide then data from each file is precedes by its file name

```
[19] !tail 3.txt.save
hi hello 12 34 23 43 45 233 2344 2345 556 3566 43456 4545 43625 234526 235246 24524
```

10. Mv

It stands for move

Syntax : !mv abc.txt d1

Description : It is used to move one or more files or directories from one place to another in a file system like linux

It has two distinct functions :

1. it renames a file or folder
2. it moves a group of files to a different directory

No space is consumed on a disk.

```
!mv 4.txt 3.txt

[30] !cat 3.txt
hi hello 12 34 23 43 45 233 2344 2345 556 3566 43456 4545 43625 234526 235246 24524
```

11. Cp

It stands for copy

Syntax : !cp abc.txt newfile.txt

Description : This command is used to copy files or group of files or directory . it creates an exact image of a file on a disk with different file name. CP command requires at least two filenames in its arguments.

```
[27] !cp 3.txt.save 4.txt

[28] !cat 4.txt
hi hello 12 34 23 43 45 233 2344 2345 556 3566 43456 4545 43625 234526 235246 24524
```

12. Wc

Syntax : !wc <filename>

Description : The wc (word count) command displays the number of lines, words, and bytes in a file. It's useful for quickly assessing the size and content of text files.

```
[20] !wc 3.txt.save
 1 18 84 3.txt.save
```

13. Date

Syntax : !date

Description : The date command prints the current date and time. It can also be used to format and manipulate date and time output in various ways.

```
[ ] !date

Fri Jun 21 05:07:39 AM UTC 2024
```

14. Passwd

Syntax : !passwd

Description : The passwd command is used to change a user's password. It prompts the user for the current password and the new password, ensuring secure password updates.

```
[31] !passwd

New password:
Retype new password:
passwd: password updated successfully
```

15. Mkdir

It stands for make directory

Syntax : !mkdir d1

Description : In linux it allows the user to create directories . This command can create multiple directories at once as well as set the permissions for the directories

```
[ ] !ls

a.out  practical-1.c  sample_data

[ ] !mkdir prac2.c

[ ] !ls

a.out  prac2.c  practical-1.c  sample_data
```

16. Rmdir

It stands for remove directory

Syntax : !rmdir d2

Description : it is used to removes a directory , specified by the directory parameter ,from the system . The directory must be empty before we remove it and we must have the write permission in its parent directory

```
[ ] !rmdir prac2.c

[ ] !ls

a.out  practical-1.c  sample_data
```

17. Hostname

Syntax : !hostname

Description : The hostname command displays the current host system's name. It can also be used to set or change the system's hostname.

```
[42] !hostname

b81b6c5e1bcd
```

18. Ps

Syntax : !ps

Description : The ps (process status) command displays information about active processes. It provides details such as process IDs, CPU usage, and the commands being executed.

```
!ps

  PID TTY          TIME CMD
    1 ?        00:00:00 docker-init
    7 ?        00:00:08 node
   17 ?        00:00:01 oom_monitor.sh
   19 ?        00:00:00 run.sh
   21 ?        00:00:01 kernel_manager_
   23 ?        00:00:00 tail
   66 ?        00:00:08 python3 <defunct>
   67 ?        00:00:01 colab-fileshim.
   85 ?        00:00:08 jupyter-noteboo
   86 ?        00:00:02 dap_multiplexer
  230 ?        00:00:21 python3
  281 ?        00:00:04 python3
 1809 ?        00:00:00 language_servic
 1814 ?        00:00:03 node
 3983 ?        00:00:01 language_servic
 3989 ?        00:00:16 node
10952 ?        00:00:00 sleep
10953 ?        00:00:00 ps
```

19. Uptime

Syntax : !uptime

Description : The uptime command shows how long the system has been running, along with the number of users and system load averages. It's useful for monitoring system stability and performance.

```
[45] !uptime

 11:49:05 up 45 min,  0 users,  load average: 0.34, 0.32, 0.33
```

20. free -h

Syntax : !free

Description : The free command provides information about the system's memory usage, including total, used, and available memory. It helps in assessing memory allocation and performance.

```
[46] !free -h

              total        used        free      shared  buff/cache   available
Mem:           12Gi       917Mi       8.0Gi       7.0Mi       3.8Gi        11Gi
Swap:            0B          0B          0B
```

21. Uname

Syntax : !uname

Description : The uname command prints system information, such as the operating system name, kernel version, and hardware architecture. It's often used for system diagnostics and troubleshooting.

```
[49] !uname -a

Linux b81b6c5e1bcd 6.1.85+ #1 SMP PREEMPT_DYNAMIC Sun Apr 28 14:29:16 UTC 2024 x86_64 x86_64 x86_64 GNU/Linux
```

PRACTICAL – 3

AIM : Using Divide and Conquer strategies design a class for Concurrent Quick sort using C++

Divide and Conquer algorithm :

Divide and Conquer Algorithm is a problem-solving technique used to solve problems by dividing the main problem into subproblems, solving them individually and then merging them to find solution to the original problem. In this article, we are going to discuss how Divide and Conquer Algorithm is helpful and how we can use it to solve problems.

Working of Divide and Conquer Algorithm:

Divide and Conquer Algorithm can be divided into three steps: **Divide**, **Conquer** and **Merge** .

1. Divide:

- Break down the original problem into smaller subproblems.
- Each subproblem should represent a part of the overall problem.
- The goal is to divide the problem until no further division is possible.

2. Conquer:

- Solve each of the smaller subproblems individually.
- If a subproblem is small enough (often referred to as the "base case"), we solve it directly without further recursion.
- The goal is to find solutions for these subproblems independently.

3. Merge:

- Combine the sub-problems to get the final solution of the whole problem.
- Once the smaller subproblems are solved, we recursively combine their solutions to get the solution of larger problem.
- The goal is to formulate a solution for the original problem by merging the results from the subproblems.

Characteristics of Divide and Conquer Algorithm:

Divide and Conquer Algorithm involves breaking down a problem into smaller, more manageable parts, solving each part individually, and then combining the solutions to solve the original problem. The characteristics of Divide and Conquer Algorithm are:

- **Dividing the Problem**: The first step is to break the problem into smaller, more manageable subproblems. This division can be done recursively until the subproblems become simple enough to solve directly.
- **Independence of Subproblems**: Each subproblem should be independent of the others, meaning that solving one subproblem does not depend on the solution of another. This allows for parallel processing or concurrent execution of subproblems, which can lead to efficiency gains.
- **Conquering Each Subproblem**: Once divided, the subproblems are solved individually. This may involve applying the same divide and conquer approach recursively until the subproblems become simple enough to solve directly, or it may involve applying a different algorithm or technique.
- **Combining Solutions**: After solving the subproblems, their solutions are combined to obtain the solution to the original problem. This combination step should be relatively efficient and straightforward, as the solutions to the subproblems should be designed to fit together seamlessly.

Quick sort :

Algo :

```
void quickSort(int a[], int first, int last)
{
   int i, j, pivot, temp;
   if (first < last)
   {
      pivot = first;
      i = first;
      j = last;
      while (i < j)
      {
         while (a[i] <= a[pivot] && i < last)
         {
            i++;
         }
         while (a[j] > a[pivot])
```

```
            {
                j--;
            }
            if (i < j)
            {
                temp = a[i];
                a[i] = a[j];
                a[j] = temp;
            }
        }
        temp = a[pivot];
        a[pivot] = a[j];
        a[j] = temp;
        quickSort(a, first, j - 1);
        quickSort(a, j + 1, last);
    }
}
```

Code :

Pseudo code :

```
%%writefile prac-3.c
#include <stdio.h>
void quickSort(int a[], int first, int last)
{
    int i, j, pivot, temp;
    if (first < last)
    {
        pivot = first;
        i = first;
        j = last;
        while (i < j)
        {
            while (a[i] <= a[pivot] && i < last)
            {
                i++;
            }
            while (a[j] > a[pivot])
            {
                j--;
            }
            if (i < j)
            {
                temp = a[i];
                a[i] = a[j];
                a[j] = temp;
            }
        }
        temp = a[pivot];
        a[pivot] = a[j];
        a[j] = temp;
        quickSort(a, first, j - 1);
        quickSort(a, j + 1, last);
    }
}
void printArray(int *A, int n)
{
    for (int i = 0; i < n; i++)
    {
        printf("%d ", A[i]);
    }
    printf("\n");
}
```

```
int main()
{
    int n;
    int a[100];
    printf("Enter number of elements required : ");
    scanf("%d", &n);
    int i, j;
    for (i = 0; i < n; ++i)
    {
        fflush(stdin);
        printf("Enter number %d : ", i + 1);
        scanf("%d", &a[i]);
    }
    printf("BEFORE SORTING :\n");
    printArray(a, n);
    quickSort(a, 0, n - 1);
    printf("AFTER SORTING :\n");
    printArray(a, n);
    return 0;
}
```

```
Writing practical-3.c

[13] !gcc practical-3.c

!./a.out

Enter number of elements required : 4
Enter number 1 : 1
Enter number 2 : 8
Enter number 3 : 33
Enter number 4 : 4
BEFORE SORTING :
1 8 33 4
AFTER SORTING :
1 4 8 33
```

Conclusion:

In this laboratory experiment, the Concurrent Quick Sort class was designed using Divide and Conquer strategies in C++. By leveraging multithreading, the algorithm achieved improved efficiency and faster execution times on multi-core processors, demonstrating the advantages of parallel processing in sorting large datasets.

PRACTICAL – 4

AIM: Write a program on an unloaded cluster for several different numbers of nodes and record the time taken in each case. Draw a graph of execution time against the number of nodes.

What is HPC cluster?

An HPC cluster, or high-performance computing cluster, is a combination of specialized hardware, including a group of large and powerful computers, and a distributed processing software framework configured to handle massive amounts of data at high speeds with parallel performance and high availability.

What are the features of HPC cluster?

- **Nodes: Each computer in an HPC cluster is called a node, and each node has its own processor, memory, and storage.**
- **Network: The nodes are connected to each other using a high-speed network, such as InfiniBand or 10 Gigabit Ethernet.**
- **Scheduler: The scheduler keeps track of resources and assigns jobs to the compute resources.**
- **Software: HPC clusters use software such as distributed processing software frameworks, compilers, debuggers, and file systems.**
- **Hardware: HPC clusters include servers, storage, and a dedicated network.**
- **Unified access: Users should be able to easily access any node in the cluster.**
- **Identity management: An identity management solution can help ensure consistency across the cluster.**

Code:

```
from numba import cuda

import numpy as np

@cuda.jit

def add_arrays(a,b,res):
```

```
 index = cuda.grid(1)
 if index < a.size:
   res[index] =a[index]+b[index]
a=np.array([11,12,13,14,15])
b=np.array([10,20,30,40,50])
result = np.empty_like(a)
d_a=cuda.to_device(a)
d_b=cuda.to_device(b)
d_result=cuda.to_device(result)
threads_per_block=256
blocks_per_grid = (a.size+threads_per_block-1)
add_arrays[blocks_per_grid, threads_per_block](d_a,d_b,d_result)
d_result.copy_to_host(result)
print(result)
```

Output:

```
from numba import cuda
import numpy as np
@cuda.jit

def add_arrays(a,b,res):
  index = cuda.grid(1)
  if index < a.size:
    res[index] =a[index]+b[index]
a=np.array([11,12,13,14,15])
b=np.array([10,20,30,40,50])
result = np.empty_like(a)
d_a=cuda.to_device(a)
d_b=cuda.to_device(b)
d_result=cuda.to_device(result)
threads_per_block=256
blocks_per_grid = (a.size+threads_per_block-1)
add_arrays[blocks_per_grid, threads_per_block](d_a,d_b,d_result)
d_result.copy_to_host(result)
print(result)
```

```
[21 32 43 54 65]
```

PRACTICAL – 5

AIM : Write a program to check task distribution using Gprof.

What is profiler ?

A profiler is a performance analysis tool utilized in high-performance computing (HPC) to identify sections of a program that consume the most computational resources, such as time or memory. Profilers are essential for optimizing and tuning HPC applications, ensuring they run as efficiently as possible.

Profiling can provide a detailed understanding of code execution from a timing perspective, enabling developers to optimize the code to handle various runtime conditions and loads. Profiling results can be ingested by compilers that support profileguided optimization, further enhancing the performance of the application.

Introduction to Gprof and its features

Gprof, short for GNU Profiler, is a performance analysis tool for Unix-based systems. It was developed by the GNU Project and is widely used for profiling applications to identify performance bottlenecks. Gprof provides detailed information on the time consumed by functions, the frequency of function calls, and the overall program execution, making it an essential tool in high-performance computing (HPC) for optimizing code and improving efficiency.

Features of Gprof in High-Performance Computing

1. Call Graph Analysis:

o Gprof generates a call graph, which illustrates the relationships between different functions in a program. This graph shows which functions call which other functions and how often they do so, providing a visual representation of the program's execution flow.

2. Flat Profile:

o The flat profile provides detailed information about the time spent in each function, including the number of times each function was called and the total time taken by each function. This helps in identifying the most time-consuming functions in the program.

3. Function Timing:

o Gprof measures the time spent in each function and its descendants. It provides both self-time (time spent in the function itself) and cumulative time (time spent in the function and the functions it calls), enabling developers to understand the timing distribution across the program.

4. Profiling Accuracy:

o Gprof uses a combination of sampling and instrumentation methods to gather profiling data. This hybrid approach helps achieve a balance between accuracy and performance overhead.

5. Ease of Use:

o Gprof is integrated into the GNU Compiler Collection (GCC), making it easy to use for developers who are already familiar with GCC. Profiling an application with Gprof typically involves compiling the program with the -pg flag and then running the application to generate profiling data.

6. Detailed Reports:

o Gprof generates detailed reports that include information on function call frequencies, execution times, and call graphs. These reports can be analyzed to identify performance bottlenecks and guide optimization efforts.

7. Support for Parallel Programs:

o While Gprof is traditionally used for serial programs, it can also be applied to parallel programs to some extent. However, for more detailed analysis of parallel and distributed applications, specialized tools like TAU or Intel VTune might be required.

8. Low Overhead:

o Gprof introduces minimal performance overhead, making it suitable for profiling HPC applications where maintaining performance is critical during the analysis phase.

Programs :

1. Simple program and steps on grpof on it

```
%%writefile simple_program.c
#include <stdio.h>

void simple_function() {
    for (int i = 0; i < 1000000; ++i) {

    }
}

int main() {

    simple_function();
```

```
    printf("Simple program executed.\n");
    return 0;
}
```

```
Overwriting simple_program.c
```

```
[ ] !ls
```

```
sample_data  simple_program.c
```

```
!gcc -Wall -pg simple_program.c -o simple_program
```

```
[ ] !ls
```

```
sample_data  simple_program  simple_program.c
```

```
[ ] !./simple_program
```

```
Simple program executed.
```

```
[ ] !ls
```

```
gmon.out  sample_data  simple_program  simple_program.c
```

```
[ ] !gprof simple_program gmon.out > simple.txt
```

```
!cat simple.txt
```

```
Flat profile:
```

```
!cat simple.txt
 no time accumulated

  %   cumulative   self              self     total
 time   seconds   seconds    calls  Ts/call  Ts/call  name
  0.00      0.00     0.00        1     0.00     0.00  simple_function

 %         the percentage of the total running time of the
time       program used by this function.

cumulative a running sum of the number of seconds accounted
 seconds   for by this function and those listed above it.

 self      the number of seconds accounted for by this
seconds    function alone.  This is the major sort for this
           listing.

calls      the number of times this function was invoked, if
           this function is profiled, else blank.

 self      the average number of milliseconds spent in this
ms/call    function per call, if this function is profiled,
           else blank.

 total     the average number of milliseconds spent in this
ms/call    function and its descendents per call, if this
           function is profiled, else blank.

name       the name of the function.  This is the minor sort
           for this listing. The index shows the location of
           the function in the gprof listing. If the index is
           in parenthesis it shows where it would appear in
           the gprof listing if it were to be printed.

Copyright (C) 2012-2022 Free Software Foundation, Inc.

Copying and distribution of this file, with or without modification,
are permitted in any medium without royalty provided the copyright
notice and this notice are preserved.

                     Call graph (explanation follows)


granularity: each sample hit covers 4 byte(s) no time propagated

index % time    self  children    called     name
                0.00    0.00       1/1           main [7]
[1]      0.0    0.00    0.00       1         simple_function [1]
-----------------------------------------------
```

2. Heavy computation program and gprof on it

```
%%writefile heavy_program.c
#include <stdio.h>

void heavy_function() {
    for (int i = 0; i < 100000000; ++i) {

    }
}

int main() {

    heavy_function();

    printf("Heavy program executed.\n");
    return 0;
}
```

```
!cat heavy.txt

Flat profile:

Each sample counts as 0.01 seconds.
  %   cumulative   self              self     total
 time   seconds   seconds    calls  ms/call  ms/call  name
100.00      0.20     0.20        1   200.00   200.00  heavy_function

 %         the percentage of the total running time of the
time       program used by this function.

cumulative a running sum of the number of seconds accounted
 seconds   for by this function and those listed above it.

 self      the number of seconds accounted for by this
seconds    function alone.  This is the major sort for this
           listing.

calls      the number of times this function was invoked, if
           this function is profiled, else blank.

 self      the average number of milliseconds spent in this
ms/call    function per call, if this function is profiled,
           else blank.

 total     the average number of milliseconds spent in this
ms/call    function and its descendents per call, if this
           function is profiled, else blank.

name       the name of the function.  This is the minor sort
           for this listing. The index shows the location of
           the function in the gprof listing. If the index is
           in parenthesis it shows where it would appear in
           the gprof listing if it were to be printed.

Copyright (C) 2012-2022 Free Software Foundation, Inc.

Copying and distribution of this file, with or without modification,
are permitted in any medium without royalty provided the copyright
notice and this notice are preserved.

                     Call graph (explanation follows)


granularity: each sample hit covers 4 byte(s) for 5.00% of 0.20 seconds

index % time    self  children    called     name
                0.20    0.00       1/1           main [2]
[1]     100.0   0.20    0.00       1         heavy_function [1]
-----------------------------------------------
                                                <spontaneous>
[2]     100.0   0.00    0.20                 main [2]
                0.20    0.00       1/1           heavy_function [1]
-----------------------------------------------
```

Conclusion :

Using Gprof in high-performance computing (HPC) environments enables developers to identify performance bottlenecks and optimize their applications effectively. By providing detailed analysis of function call frequencies, execution times, and generating call graphs, Gprof helps pinpoint resource-intensive sections of the code. This profiling tool, integrated with the GNU Compiler Collection (GCC), offers a straightforward workflow with minimal performance overhead, making it suitable for both serial and, to some extent, parallel programs. Overall, Gprof is an invaluable asset for enhancing the efficiency and scalability of HPC applications, ensuring they run optimally under various runtime conditions and loads.